Google falls to second place...
In the worldwide smart speaker market. They lost their spot to China's, Baidu, who grew a whopping 3,700%! https://dgit.in/sep19-84

Hush Google Assistant!
A new feature is coming to Google Assistant which will allow users to turn off speech output. https://dgit.in/sep19-85

```
printWithDelay(String message)
async {
  await Future.
delayed(oneSecond);
  print(message);
}
```

They also make it easier to read the code. Here's an example:

```
Future<void> createDescriptio
ns(Iterable<String> objects)
async {
  for (var object in objects)
{
    try {
      var file =
File('$object.txt');
      if (await file.exists())
{
        var modified = await
file.lastModified();
        print(
            'File for $object
already exists. It was modi-
fied on $modified.');
        continue;
      }
      await file.create();
      await file.
writeAsString('Start describ-
ing $object in this file.');
    } on IOException catch (e)
{
      print('Cannot create de-
scription for $object: $e');
    }
  }
}
```

WHY YOU'LL LOVE DART

Dart's Stateful Hot Reload

If you've used React Native, then you should know what Hot Reloading is. The philosophy behind hot reloading is to inject newer versions of files while your app is still running. Since the state of the running app isn't affected, this can be effectively termed Stateful Hot Reloading.

Think of it in this manner. You're building an app in Flutter and you've got this UI element that isn't functioning exactly in the manner you wanted it to. Usually, you'd stop the run. Get back to the function where you'd initiated the UI element and try to figure what went wrong.

With Dart, you can keep the app running and mess around with the UI element and see the changes reflected, all in real time. If it weren't stateful, and you made any changes while the app is running, the module you made changes to would ideally reload. And thus, you'd lose the state. This video should make it clearer - https://dgit.in/dart1

We can easily say that after you've experienced Stateful Hot Reloading, you would not want to go back.

Less jank

One of the irritating things you'd come across while browsing the web or using apps is when the UI stutters and the loading of the elements. This erratic behaviour leaves a negative impression and affects user experience. This is called Jank. And Dart has several features that inform the developer of things that might lead to more jank, thus, giving them an opportunity to make the app UI smoother. The objective is to ensure that you are rendering the same number of frames as the refresh rate of the display on which the app is running.

No more shared resources

One of the reasons that you get jank is if the garbage collection process is impeded by any other process. Let's say if there's a thread running that's utilising a shared resource, it will most certainly lock the shared resource down so that race conditions are avoided. And when the garbage collector can't do it's job because of the lock being present, you will get some jank. Dart resolves this by not using shared resources in the first place. Threads in

*DEV NEWS

*Anime4K

Gone are the days of watching Anime in low-res. Anime 4K is a high-quality real-time anime upscaling algorithm that can be implemented in any language.
https://dgit.in/2LcS9fY

*Bitbucket bids Mercurial goodbye

Given that Git has grown in popularity over recent years, Bitbucket is sunsetting support for Mercurial in favour of Git.
https://dgit.in/2ZueAme

*Dancing merge-sort

You've no doubt learnt about merge sort in school or college but have you ever had it taught to you through a song and dance routine?
https://dgit.in/2Zjgj2s